REAL-TIME E-COMMERCE DATA PLATFORM

A production-grade, hire-worthy portfolio project for senior Data Engineer roles

Kafka • Spark • Delta Lake • dbt • Airflow • Terraform • Grafana

8+	5	3	100%
Technologies	Pipeline Layers	Cloud Providers	Open Source

What interviewers will see when they open your GitHub:

-to-End System | Real Streaming | Production Patterns | IaC + CI/CD | Monitoring | dbt + Star Sche

WHY THIS PROJECT GETS YOU HIRED

Most DE portfolios are: *"I loaded a CSV into Postgres."* This project is different. It covers every pillar that hiring managers at top companies actually interview on — streaming ingestion, medallion lakehouse, distributed transformation, orchestration, infrastructure as code, and observability. You can talk about every design decision for 30 minutes.

What makes it stand out

- Real-time + batch in one system
- Full medallion lakehouse (Bronze/Silver/Gold)
- ACID transactions via Delta Lake
- Schema evolution handled correctly
- CI/CD pipeline for dbt models
- Infra provisioned with Terraform

Interview talking points you unlock

- Why Kafka over SQS / Pub-Sub?
- How do you handle late-arriving data?
- How does Delta Lake guarantee idempotency?
- Explain your SCD Type 2 approach
- How would you scale this to 10x traffic?

Technologies — each one is a resume bullet

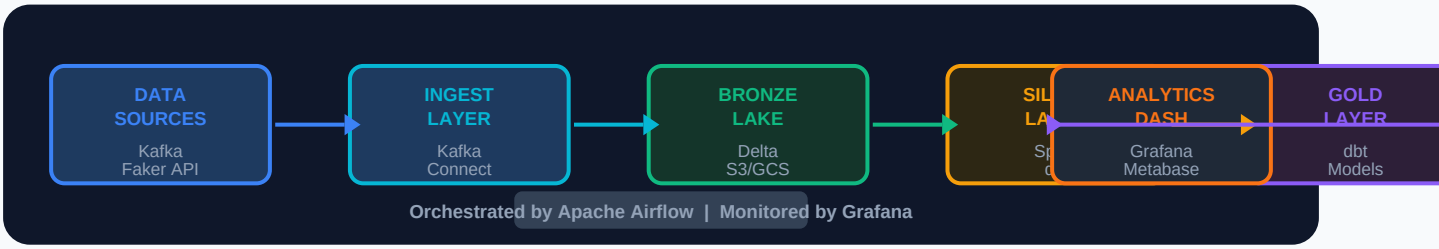
- Apache Kafka (streaming backbone)
- Apache Spark + PySpark (distributed transform)
- Delta Lake (ACID lakehouse storage)
- dbt (SQL transformations + lineage)
- Apache Airflow (orchestration + scheduling)
- Terraform (infra as code)
- Docker + docker-compose (local dev)
- Grafana + Prometheus (observability)
- Great Expectations (data quality)
- GitHub Actions (CI/CD)

Roles it targets

- Data Engineer (all seniority levels)
- Senior / Staff Data Engineer
- Analytics Engineer
- Platform / Infrastructure DE

SYSTEM ARCHITECTURE

THE PIPELINE — End to End



DATA SOURCES

- Faker-based Python producer simulates e-commerce events: orders, clicks, payments, returns
- Produces ~500 events/sec to Kafka topics (configurable)
- Schema defined in Avro + Schema Registry to enforce contracts
- Also pulls static CSVs: product catalog, customer master (batch)

KAFKA INGESTION

- 3 Kafka topics: ecom.orders, ecom.clickstream, ecom.payments
- Kafka Connect JDBC Sink for writing raw to Postgres (audit trail)
- Partitioned by customer_id for ordering guarantees
- Schema Registry prevents breaking schema changes from reaching consumers

BRONZE LAYER (Raw Delta)

- Spark Structured Streaming reads from Kafka, writes Avro-decoded JSON to Delta tables
- Append-only — never modify Bronze. This is your audit log.
- Partitioned by event_date for efficient pruning
- Schema auto-evolution enabled for non-breaking additions

SILVER LAYER (Cleaned Delta)

- Spark batch job (Airflow-triggered) deduplicates using Delta MERGE INTO
- Type casting, null handling, PII masking (customer email hashed)
- Data quality checks via Great Expectations — fail pipeline on critical violations
- SCD Type 2 for customer dimension (tracks address changes over time)

GOLD LAYER (dbt Models)

- dbt project with sources, staging, intermediate, marts layers
- Star schema: fact_orders, dim_customer, dim_product, dim_date
- Incremental models for large fact tables (only process new data)
- dbt tests: unique, not_null, accepted_values, custom revenue assertions
- dbt docs generate — auto-documented data catalog

ANALYTICS + OBSERVABILITY

- Grafana dashboards: orders/min, revenue by category, funnel drop-off
- Airflow DAG success rates, task durations, SLA misses
- Prometheus scrapes custom pipeline metrics (row counts, lag, quality scores)
- Alert rules: data freshness > 30min, quality score < 95%, pipeline failure

PROJECT STRUCTURE & CODEBASE

GitHub Repository Layout

```
# ecom-data-platform/
■■■ infra/                    # Terraform IaC
■   ■■■ main.tf               # Cloud resources (S3/GCS/ADLS)
■   ■■■ kafka.tf              # MSK / Confluent cluster
■   ■■■ compute.tf            # EMR / Dataproc cluster
■   ■■■ variables.tf
■■■ ingestion/
■   ■■■ producer/             # Faker event producer
■   ■   ■■■ ecom_producer.py
■   ■■■ kafka_connect/        # Connector configs (JSON)
■■■ spark/
■   ■■■ bronze/               # Streaming: Kafka -> Delta Bronze
■   ■   ■■■ kafka_to_bronze.py
■   ■■■ silver/               # Batch: dedup + clean Bronze -> Silver
■   ■   ■■■ orders_silver.py
■   ■   ■■■ customers_scd2.py
■   ■■■ utils/                # Schema, spark session helpers
■■■ dbt_project/              # dbt Gold layer
■   ■■■ models/
■   ■   ■■■ staging/
■   ■   ■■■ intermediate/
■   ■   ■■■ marts/            # fact_orders, dim_customer, etc.
■   ■■■ tests/
■   ■■■ dbt_project.yml
■■■ airflow/
■   ■■■ dags/
■   ■   ■■■ bronze_streaming_monitor.py
■   ■   ■■■ silver_processing.py
■   ■   ■■■ gold_dbt_run.py
■   ■■■ plugins/
■■■ quality/
■   ■■■ expectations/         # Great Expectations suites
■■■ monitoring/
■   ■■■ grafana/              # Dashboard JSON exports
■   ■■■ prometheus/           # Alert rules YAML
■■■ docker-compose.yml        # Full local stack
■■■ Makefile                  # make up, make test, make dbt-run
■■■ .github/workflows/        # CI/CD pipelines
■   ■■■ dbt_ci.yml
■   ■■■ spark_tests.yml
■■■ README.md                 # Architecture diagram + setup guide
```

KEY CODE — WHAT TO BUILD

1. Kafka Event Producer (ingestion/producer/ecom_producer.py)

```
from confluent_kafka import Producer
from confluent_kafka.schema_registry.avro import AvroSerializer
from faker import Faker
import json, time, random

# Produces realistic e-commerce events at configurable throughput
def produce_order_event(producer, schema_str, topic):
    fake = Faker()
    event = {
        "order_id":    str(uuid.uuid4()),
        "customer_id": fake.uuid4(),
        "product_id":  random.choice(PRODUCT_IDS),
        "amount":      round(random.uniform(9.99, 499.99), 2),
        "status":      random.choice(["placed","confirmed","shipped"]),
        "event_ts":    int(time.time() * 1000),
    }
    producer.produce(topic, value=avro_serialize(event, schema_str))
    producer.poll(0)
```

2. Bronze Streaming Job (spark/bronze/kafka_to_bronze.py)

```
from delta import configure_spark_with_delta_pip
from pyspark.sql import SparkSession
from pyspark.sql.functions import from_json, col, current_timestamp
# Reads Kafka -> decodes Avro -> writes append-only Delta Bronze
spark = configure_spark_with_delta_pip(SparkSession.builder
    .appName("KafkaToBronze")
    .config("spark.sql.extensions","io.delta.sql.DeltaSparkSessionExtension")
    .getOrCreate())
df = (spark.readStream.format('kafka')
    .option('kafka.bootstrap.servers', BROKER)
    .option('subscribe', 'ecom.orders')
    .load()
    .select(from_json(col('value').cast('string'), ORDER_SCHEMA).alias('d'))
    .select('d.*')
    .withColumn('ingested_at', current_timestamp())
    .withColumn('event_date',  col('event_ts').cast('date')))
query = (df.writeStream.format('delta')
    .outputMode('append')
    .option('checkpointLocation', CHECKPOINT_PATH)
    .partitionBy('event_date')
    .start(BRONZE_PATH))
```

3. Silver MERGE (dedup + upsert) — spark/silver/orders_silver.py

```
from delta.tables import DeltaTable
# Idempotent upsert from Bronze -> Silver
# Re-running this for same date is safe (MERGE handles duplicates)
bronze_df = (spark.read.format('delta').load(BRONZE_PATH)
    .filter(col('event_date') == processing_date)
    .dropDuplicates(['order_id'])       # dedup within batch
    .withColumn('email_hash',            # PII masking
        sha2(col('customer_email'), 256)))
silver = DeltaTable.forPath(spark, SILVER_PATH)
silver.alias('s').merge(
    bronze_df.alias('b'),
    's.order_id = b.order_id'
).whenMatchedUpdateAll()
 .whenNotMatchedInsertAll()
 .execute()
```

4. dbt Gold Model (models/marts/fact_orders.sql)

```
-- models/marts/fact_orders.sql
-- Incremental model: only processes new orders each run
{{
    config(
        materialized = 'incremental',
        unique_key   = 'order_id',
        on_schema_change = 'merge_new_columns'
    )
}}

SELECT
    o.order_id,
    o.customer_id,
    o.product_id,
    d.date_key,
    o.amount                    AS gross_revenue,
    o.amount * 0.85             AS net_revenue,
    o.status,
    o.event_ts                  AS order_timestamp
FROM {{ ref('stg_orders') }}    o
JOIN {{ ref('dim_date') }}       d ON DATE(o.event_ts) = d.full_date
{% if is_incremental() %}
WHERE o.event_ts > (SELECT MAX(order_timestamp) FROM {{ this }})
{% endif %}
```

5. Airflow DAG — Gold Layer Orchestration

```
from airflow.decorators import dag, task
from airflow.providers.apache.spark.operators.spark_submit import SparkSubmitOperator
from airflow.operators.bash import BashOperator
from pendulum import datetime

@dag(schedule='0 * * * *', start_date=datetime(2024,1,1), catchup=False)
def gold_pipeline():

    silver_job = SparkSubmitOperator(
        task_id='run_silver_processing',
        application='spark/silver/orders_silver.py',
        conn_id='spark_default')

    dq_checks = BashOperator(
        task_id='great_expectations_validate',
        bash_command='ge checkpoint run orders_silver_checkpoint')

    dbt_run = BashOperator(
        task_id='dbt_run_gold',
        bash_command='dbt run --select marts --target prod')

    dbt_test = BashOperator(
        task_id='dbt_test_gold',
        bash_command='dbt test --select marts')

    silver_job >> dq_checks >> dbt_run >> dbt_test

gold_pipeline()
```

STEP-BY-STEP IMPLEMENTATION PLAN

01 **Set up local environment with Docker Compose**

Spin up: Kafka + Zookeeper, Schema Registry, Kafka UI, Postgres, Spark (standalone), Airflow, MinIO (local S3), Grafana, Prometheus. All services in one docker-compose.yml.

```
make up # Starts entire stack in ~2 min
```

02 **Write the Kafka event producer**

Build ecom_producer.py using Faker. Define Avro schemas for orders, clickstream, payments. Register schemas in Schema Registry. Produce 500 events/sec and verify in Kafka UI.

```
python producer/ecom_producer.py --rate 500 --topic ecom.orders
```

03 **Build the Bronze Spark Streaming job**

PySpark Structured Streaming reads from Kafka topic, deserializes Avro with Schema Registry, writes partitioned Delta tables to MinIO. Test with a 5-minute sample run.

```
spark-submit spark/bronze/kafka_to_bronze.py
```

04 **Build the Silver batch jobs**

Write orders_silver.py (MERGE dedup + PII masking) and customers_scd2.py (SCD Type 2 via Delta). Add Great Expectations validation suite that checks for nulls, counts, and value ranges.

```
spark-submit spark/silver/orders_silver.py --date 2024-01-15
```

05 **Build the dbt Gold project**

Create staging models (stg_orders, stg_customers, stg_products), intermediate joins, and mart models (fact_orders, dim_customer, dim_product, dim_date). Add schema tests and a custom revenue test.

```
dbt run --select marts && dbt test --select marts
```

06 **Wire everything with Airflow DAGs**

Create 3 DAGs: bronze_monitor (watches streaming health), silver_processing (hourly batch), gold_dbt_run (runs dbt after silver). Set up email/Slack alerting on failure.

```
airflow dags trigger gold_pipeline
```

07 **Provision cloud infra with Terraform**

Write Terraform modules for S3/GCS bucket, IAM roles, Kafka cluster (MSK or Confluent Cloud free tier), and Spark cluster. Store state in S3 backend. Use variables for dev/prod.

```
terraform plan -var-file=dev.tfvars && terraform apply
```

08 **Set up CI/CD with GitHub Actions**

Two workflows: (1) dbt_ci.yml — on every PR, run dbt compile + dbt test on modified models. (2) spark_tests.yml — run pytest on Spark transformation unit tests.

```
git push origin feature/new-model # triggers CI automatically
```

09 **Build Grafana dashboards**

Import dashboard JSON for: (1) Pipeline Health — DAG success rate, task duration, SLA status. (2) Business Metrics — orders/minute, revenue by category, funnel conversion. Set up Prometheus alert rules.

```
http://localhost:3000 # Grafana UI after make up
```

10 **Write a killer README**

Architecture diagram (use draw.io or Excalidraw), local setup in 5 commands, explanation of design decisions (why Kafka over SQS, why Delta over Parquet, why dbt incremental), screenshots of Grafana dashboards.

```
# Setup: git clone -> cp .env.example .env -> make up -> make seed
```

DATA MODEL — STAR SCHEMA (GOLD LAYER)

The Gold layer exposes clean, analytics-ready tables in a star schema. dbt owns all transformations from Silver to Gold with full lineage tracking.

fact_orders (incremental)

```
order_id VARCHAR PK
customer_key INT FK -> dim_customer
product_key INT FK -> dim_product
date_key INT FK -> dim_date
gross_revenue DECIMAL
net_revenue DECIMAL
quantity INT
status VARCHAR
order_timestamp TIMESTAMP
```

dim_customer (SCD Type 2)

```
customer_key INT PK (surrogate)
customer_id VARCHAR NK (natural key)
full_name VARCHAR
email_hash VARCHAR (PII masked)
city VARCHAR
country VARCHAR
segment VARCHAR
valid_from DATE
valid_to DATE (NULL = current)
is_current BOOLEAN
```

dim_product (Type 1)

```
product_key INT PK
product_id VARCHAR NK
product_name VARCHAR
category VARCHAR
sub_category VARCHAR
unit_price DECIMAL
supplier_id VARCHAR
```

dim_date (static)

```
date_key INT PK
full_date DATE
day_of_week INT
week_of_year INT
month_num INT
month_name VARCHAR
quarter INT
year INT
is_weekend BOOLEAN
```

INTERVIEW PREP — QUESTIONS THIS PROJECT ANSWERS

Q1. Why Kafka instead of a REST API or a message queue like SQS?

Kafka provides durable, replayable, ordered event logs with consumer group semantics. Unlike SQS (at-most-once), Kafka enables multiple independent consumers (Spark, Connect, monitoring) from the same stream. Replayability means I can reprocess Bronze from any offset if a bug is found downstream.

Q2. How do you handle late-arriving data in your streaming pipeline?

I use Spark Structured Streaming watermarks (withWatermark('event_ts', '2 hours')). Events up to 2 hours late are still aggregated correctly. Beyond that, they're dropped with a dead-letter queue to S3 for manual investigation. The Bronze layer is append-only so late raw data is always preserved regardless.

Q3. How do you guarantee exactly-once processing in your pipeline?

Three layers: (1) Kafka consumer with manual offset commits after successful write. (2) Spark Structured Streaming's checkpointing for fault-tolerant micro-batches. (3) Delta Lake MERGE INTO for Silver — even if the Spark job runs twice for the same date, the MERGE is idempotent so duplicates never reach Silver or Gold.

Q4. Walk me through your SCD Type 2 implementation.

I use dbt snapshots with strategy: timestamp on updated_at. When a customer changes their address, dbt inserts a new row with the new values and a new valid_from date, and closes the old row by setting valid_to and is_current=false. fact_orders always joins to dim_customer on customer_key (surrogate), so historical orders retain the address that was active at order time.

Q5. How would you scale this pipeline to handle 10x traffic?

Kafka: increase partition count from 12 to 120 (linear parallelism). Spark: increase executor count; the stateless Bronze streaming scales horizontally. Delta: already handles concurrent writes via optimistic concurrency. dbt Gold: incremental models only process new data, so cost grows linearly not exponentially. Terraform makes this a config change — just increase cluster size variables.

Q6. What data quality checks do you run and how do pipeline failures work?

Great Expectations validates Silver output: row counts didn't drop >15% from yesterday, no nulls in order_id/customer_id, revenue is always positive, status is in accepted values. A failed checkpoint raises an exception that Airflow catches, marks the DAG as failed, sends a Slack alert, and stops Gold from running on stale data.

TOOLS, CLOUD OPTIONS & README TEMPLATE

Local Development (Free — Start Here)

- Docker Desktop + docker-compose
- MinIO — local S3-compatible object store
- Kafka in KRaft mode (no Zookeeper needed)
- Apache Spark 3.5 standalone
- Airflow 2.9 with LocalExecutor
- DuckDB for lightweight SQL queries on Delta
- Grafana + Prometheus (containerized)

AWS Stack (Free Tier Eligible)

- S3 — Delta Lake storage
- MSK (Managed Kafka) — or use Confluent Cloud free
- EMR Serverless — Spark jobs (pay per run)
- MWAA or EC2-hosted Airflow
- RDS Postgres — metadata store
- Redshift Serverless — optional Gold serving layer
- Terraform AWS provider

GCP Stack (Free Tier Eligible)

- GCS — Delta Lake storage
- Pub/Sub — or Confluent Cloud on GCP
- Dataproc Serverless — Spark jobs
- Cloud Composer (Airflow managed)
- BigQuery — Gold layer serving
- Cloud Monitoring — metrics + alerts
- Terraform Google provider

Tips for Cost Control

- Always use Spot/Preemptible instances for Spark
- Set auto-shutdown on EMR/Dataproc after job
- Use Confluent Cloud free tier (1M events/month)
- Redshift/BigQuery have free query tiers
- Destroy Terraform resources when not testing
- Budget alerts in AWS/GCP — set $10 limit

README.md Template (copy this to your GitHub)

```
# Real-Time E-Commerce Data Platform

## Architecture

![Architecture Diagram](docs/architecture.png)

## Tech Stack

| Layer | Technology | Purpose |
|-------|-----------|---------|
| Ingestion | Apache Kafka + Schema Registry | Event streaming |
| Bronze | Spark Structured Streaming + Delta Lake | Raw storage |
| Silver | PySpark + Delta MERGE | Dedup + quality |
| Gold | dbt (incremental models) | Analytics models |
| Orchestration | Apache Airflow | Scheduling + monitoring |
| IaC | Terraform | Cloud provisioning |
| Quality | Great Expectations | Data validation |
| Monitoring | Grafana + Prometheus | Observability |

## Quick Start (Local)

```bash
git clone https://github.com/yourname/ecom-data-platform
cd ecom-data-platform
cp .env.example .env          # fill in your config
make up                        # starts all services
make seed                      # starts event producer
make dbt-run                   # runs Gold layer
open http://localhost:3000      # Grafana dashboard
```

Key Design Decisions

- **Why Delta Lake?** ACID transactions + time travel + schema evolution
- **Why Kafka?** Replayable event log, multiple consumer groups
- **Why incremental dbt?** Process only new data — O(new rows) not O(total rows)
- **Why SCD Type 2?** Preserve historical customer context for order analysis
```

You now have everything to build a portfolio project that will impress any data engineering interviewer. Build it, push it to GitHub, write the README, and you will stand out from 95% of candidates. Good luck.